Set is a mutable data type that contains an unordered collection of unique items. A set is represented with { }.

Every element in the set should be unique (no duplicates) and must be immutable (which cannot be changed). But the set itself is mutable. We can add or remove items / elements from it.

```
#Python Set
set1 = {3, 4, 5, 1, 2, 5}
print(set1)
```

```
{1, 2, 3, 4, 5}
```

```
#Operations on Set
set1.add(9) #Add Element to Set
print(set1)
set1.update([3, 10]) #Update existing Set with new elements
print(set1)
set1.discard(2) #Removes an element from the Set (No Error if element does not exists, Set remains unchanged)
print(set1)
set1.remove(5) #Removes an element from the Set (Raises an error in element does not exists)
print(set1)
set1.pop() #removes random element
print(set1)
set1.clear() #Removes all elements from the Set
print(set1)
```

```
{1, 2, 3, 4, 5, 9}
{1, 2, 3, 4, 5, 9, 10}
{1, 3, 4, 5, 9, 10}
{1, 3, 4, 9, 10}
{3, 4, 9, 10}
set()
```

```
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5}
print(set1|set2)  #creates a new set that contains all the elements that are present in either set1 or set2, or in both se
print(set1.union(set2))
```

```
{1, 2, 3, 4, 5}
{1, 2, 3, 4, 5}
```

```
set1 = {11, 12, 13, 14}
set2 = {13, 14, 15}
print(set1.union(set2))
```

```
{11, 12, 13, 14, 15}
```

```
set1
```

```
{11, 12, 13, 14}
```

```
print(set1&set2)  ##creates a new set that contains only the elements that are present in both set1 and set2

print(set1.intersection(set2))
```

```
{13, 14}
{13, 14}
```

```
print(set1-set2)
print(set1.difference(set2))  #it returns all elements from set1 that are not present in set2
```

```
{11, 12}
{11, 12}
```

```
set1
```

```
{11, 12, 13, 14}
```

```
print(set1^set2)  # it returns all elements that are exclusive to either set1 or set2, excluding the elements that are com
print(set1.symmetric_difference(set2))
```

```
{11, 12, 15}
{11, 12, 15}
```

Dictionaries

- List of key-value pairs { }
- Unordered collections of objects, not indexed
- Store objects in a random order to provide faster lookup
- Data type are heterogeneous
- Element are accessed by a keyword, not index
- Elements are mutable
- dict = {"key1”: value1, “key2”: value2}

Dictionary is an unordered collection of 'key':'value' pairs. It is generally used when we have a huge amount of data. Dictionaries are optimized for retrieving data.

```
#Python Dictionary
dict1 = {1:'value', 'key':2}
print(type(dict1))
```

```
<class 'dict'>
```

```
dict1['key']
```

```
2
```

```
#Create Dictionary in Python
dict2={"Rajesh":"Faculty", "SIT":"College"}
print(dict2["Rajesh"])
print(dict2["SIT"])
```

```
Faculty
College
```

```
#Update value of a key in Python Dictionary
dict2["Rajesh"]="Asst. Prof."
print(dict2["Rajesh"])
dict2
```

```
Asst. Prof.
{'Rajesh': 'Asst. Prof.', 'SIT': 'College'}
```

```
#Add new element to the existing Python Dictionary
dict2["Amit"]="Asst. Prof."
print(dict2["Amit"])
print(dict2)
```

```
Asst. Prof.
{'Rajesh': 'Asst. Prof.', 'SIT': 'College', 'Amit': 'Asst. Prof.'}
```

```
dict3 = sorted(dict2)
print(dict3)
print(dict2.values())
dict2.keys()
type(dict3)
```

```
['Amit', 'Rajesh', 'SIT']
dict_values(['Asst. Prof.', 'College', 'Asst. Prof.'])
list
```

```
for i in sorted(dict2):
  print((i, dict2[i]), end = "#")
 # print('{0} : {1}'.format(i, dict2[i])) #The format() method in Python is used to format strings dynamically. It allows
  #strings with placeholders,which are then replaced by the values of specified variables or expressions.
```

```
('Amit', 'Asst. Prof.')#('Rajesh', 'Asst. Prof.')#('SIT', 'College')#
```

```
template = "Welcome, {name}! Your balance is ${balance:.2f}."
formatted_string = template.format(name="Riya", balance=123.456)
print(formatted_string)
```

```
Welcome, Riya! Your balance is $123.46.
```

```
#Input/Output in Python
#print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False) #The stream is forcibly flushed If flush is True. Defaul
#The job of the flush is to make sure that the output, which is being buffered, goes to the destination safely
print(1,2,3,4)
print(1,2,3,4, sep='#', end='$')
print(1,2,3,4, sep=' ', end='\n')
print(1,2,3,4, sep='#', end='$', flush=False)
```

```
1 2 3 4
1#2#3#4$1 2 3 4
1#2#3#4$
```

```
# open and read the file
my_file = open("abc.txt","r")
my_file = open("abc.txt","w")
print('SIT', 'Nagpur', file=my_file)
# print the contents of the file
my_file = open("abc.txt","r")
print(my_file.read())
```

```
SIT Nagpur
```

```
from time import sleep

for second in range(3, 0, -1):
    print(second)
    sleep(1)
print("Go!")
```

```
3
2
1
Go!
```

```
from time import sleep

for second in range(3, 0, -1):
    print(second, end=" ")
    sleep(1)
print("Go!")
```